FRANCESCA ROSSI · 26/10/2026

I TUOI AGENTI IN LOCALE

Come funziona OpenCode

Legge il tuo codice, esegue comandi e modifica file.
Sul tuo computer, con le tue regole.

$ opencode

↓ PER INIZIARE

Il modello propone, l'ambiente esegue

Quattro passaggi, ripetuti fino a quando il compito è finito.

01 →

Prompt

Chiedi in linguaggio naturale: «aggiungi i test».

02 →

Modello

Sceglie il prossimo passo e i tool da usare.

03 →

Tool

Legge, cerca, esegue comandi, modifica file.

04 →

Risultato

L'output rientra nel contesto, e si riparte.

↳ Il risultato diventa contesto per il prossimo giro — fino a «fatto».

Un server, molti volti

Il terminale è solo un client. Lo stesso motore alimenta desktop, IDE e SDK.

SERVER LOCALE

Sessioni
Storico, messaggi, stato

Tool
Esecuzione e permessi

Provider
Modelli e streaming

LSP + MCP
Codice e strumenti esterni

HTTP · SSE · OpenAPI 3.1

CLIENT

TUI	Terminale
APP	Desktop
IDE	Estensione editor
SDK	HTTP · script

↳ Tutto gira sulla tua macchina: i tuoi file, le tue chiavi, i tuoi processi.

Il modello non tocca i file

Ogni tool è una funzione con nome, descrizione e schema JSON degli argomenti.
Il modello emette la chiamata; OpenCode la esegue e rimette l'output nel contesto.

01 **Il modello riceve lo schema**

Nome, descrizione e argomenti validi per ogni tool disponibile.

02 **Emette una tool call**

Esempio: `read({ filePath: "src/auth.ts" })`. Nessun accesso diretto al disco.

03 **OpenCode esegue**

Controlla i permessi, esegue il tool, restituisce output ed exit code.

Lo schema è il contratto: se un argomento non è valido, la chiamata non parte.

read
legge un file

grep
cerca nel testo

glob
elenca per glob

edit
modifica chirurgica

write
crea o sovrascrive

bash
esegue un comando

webfetch
scarica una pagina

task
delega a un subagente

↳ allow / ask / deny decidono cosa può passare: la configurazione è nella prossima slide. — docs /tools

Permessi: allow, ask, deny

Le regole stanno in opencode.json.
Si valutano in ordine e vince l'ultima che matcha.
Dentro ogni tool, metti il catch-all `"*"` per primo, poi le eccezioni.

OPENCODE.JSON · PERMISSION

```
{
  "$schema": "https://opencode.ai/config.json",
  "permission": {
    "*": "ask",
    "bash": { "*": "ask", "git *": "allow", "rm *": "deny" },
    "edit": "allow",
    "webfetch": "deny"
  }
}
```

`allow`

esegue senza chiedere

`ask`

ferma e chiede: once / always / reject

`deny`

blocca l'azione: non viene eseguita

`--auto` approva tutto tranne i deny.

Permessi globali e di agente si fondono: vince l'agente.

Default: allow. `doom_loop` e `external_directory` = ask; i file `.env` = ask.

↳ Il vecchio tools booleano è deprecato dalla v1.1.1: la fonte di verità è permission. — docs /permissions

Profili, non programmi diversi

Un agente è un profilo: prompt di sistema, tool disponibili e permessi.
Cambia il profilo, non il motore.

`primary` → la conversazione che vedi `subagent` → sessione isolata, solo il risultato

PRIMARY · DEFAULT

Build

Legge, scrive, esegue comandi. Tutti i tool, con i permessi che decidi tu.

`edit · write · bash`

PRIMARY · PIANIFICAZIONE

Plan

Analizza e propone un piano. Modifiche ai file negate, tranne i piani; bash non è ristretto.

`read · grep · bash`

SUBAGENT · ISOLATI

Subagenti

General porta a termine task lunghi; Explore cerca nel codice. Lavorano in una sessione figlia.

`general · explore`

↳ Tab cambia agente primario. I subagenti si creano e si invocano nella prossima slide. — docs /agents

Subagenti: si scrivono in markdown

Un file per agente: la description decide quando il primario delega.

.OPENCODE/AGENTS/REVIEW.MD

```
---
description: Code review senza modifiche
mode: subagent
model: anthropic/claude-sonnet-4-5
permission:
  edit: deny
  bash: ask
---
Sei un reviewer severo. Leggi il diff,
elenca i rischi e proponi i test mancanti.
```

AUTOMATICO

Il primario legge le description e delega con il tool `task` quando il task combacia.

MANUALE

`@review controlla il diff` lo invoca subito.
`Tab` cambia agente primario.

DOVE

`.opencode/agents/` nel repo, `~/.config/opencode/agents/` globale. Il nome del file è il nome: `review.md` → `@review`.

CLI

`opencode agent create` · `opencode agent list`

`permission.task` decide quali subagenti un agente può chiamare. Senza `model`, eredita quello del primario.

↳ Il subagente lavora in una sessione figlia: torna solo il risultato, non il suo transcript. — docs /agents

Opzione facile per creare un sub-agent

`opencode agent create`: description, mode, permessi, modello. Poi scrive il markdown.

COSA CHIEDE

`description`	cosa deve fare; il primary la usa per delegare.
`mode`	primary, subagent o all.
`permissions`	solo quelle scelte; il resto è deny.
`model`	opzionale: provider/model.

Scrive in `.opencode/agents/` · globale in `~/.config/opencode/agents/`.

OPPURE NON INTERATTIVO

```
opencode agent create \
  --path .opencode \
  --description "Code review senza modifiche" \
  --mode subagent \
  --permissions read,grep,glob
```

```
PS C:\Users\cesch> opencode agent create

┌  Create agent
│
◇  Description
│  summarize text
│
◇  Agent text-summarizer generated
│
◆  Select permissions to allow (Space to toggle)
│  ■ bash
│  ■ read
│  ■ edit
│  ■ glob
│  ■ grep
│  ■ webfetch
│  ■ task
│  ■ todowrite
│  ■ websearch
│  ■ lsp
│  ■ skill
└
```

`opencode agent create` — procedura guidata

Markdown normale: versionalo e correggilo a mano.

↳ Il resto è deny; `opencode agent list` mostra gli agenti caricati. — docs /agents · /cli

Il contesto è la memoria di lavoro

Il modello non ha accesso al disco: conosce solo i token nella finestra di contesto. Ogni richiesta rimanda al provider il transcript, non il repo.

01 **System prompt**
Regole di comportamento, tool disponibili, ambiente.

02 **AGENTS.md**
Le convenzioni del progetto, scritte da te con /init.

03 **Output dei tool**
Contenuto dei file, risultati di ricerca, stdout: tutto ciò che i tool hanno restituito.

04 **Diagnostica LSP**
Errori e tipi, come li vedrebbe il tuo editor.

QUANDO SI RIEMPIE

/compact

Riassume la conversazione e riparte con l'essenziale.

La cronologia completa resta su disco.

↳ Se non è nella finestra, non esiste: il resto va riletto, riassunto o chiesto di nuovo.

Dove si configura

I file si fondono, non si sostituiscono: il progetto vince sul globale solo per le chiavi che tocca.

OPENCODE.JSONC · ROOT DEL PROGETTO

```
{
  "$schema": "https://opencode.ai/config.json",
  "model": "anthropic/claude-sonnet-4-5",
  "instructions": ["docs/conventions.md"],
  "provider": {
    "anthropic": { "options": { "apiKey": "{env:ANTHROPIC_API_KEY}" } }
  }
}
```

NEL REPO

`opencode.json`	modello, permessi, agent, provider
`AGENTS.md`	convenzioni; si crea con /init
`.opencode/agents/`	un file markdown per agente
`.opencode/commands/`	comandi slash riutilizzabili
`.opencode/plugins/`	hook e integrazioni

PRECEDENZA

vince l'ultimo che definisce la chiave

01 · remoto	02 · globale	03 · custom	04 · progetto	05 · .opencode/	06 · inline
.well-known/opencode	~/.config/opencode/	$OPENCODE_CONFIG	./opencode.json	agents · commands	$OPENCODE_CONFIG_CONTENT

↳ Globale in ~/.config/opencode · progetto nella root · variabili `{env:KEY}` e `{file:./path}` . — docs /config

Quattro punti di innesto

Si configura, non si riscrive: provider, MCP, plugin e comandi.

mcp

MCP server

Collega strumenti esterni: database, tracker, browser.

provider

Qualunque modello

Anthropic, OpenAI, Google, locali. BYOK: le tue chiavi.

plugin · skill

Procedure riutilizzabili

Impacchetta flussi e hook, condividili col team.

.opencode/command

Comandi custom

Scorciatoie in markdown: /review, /release.

↳ BYOK = bring your own key: le chiavi le fornisci tu, provider per provider.

Tre cose da portarsi via

01	**È un agente, non un autocompletamento**	Guarda, agisce, verifica — entro i permessi che dai.
02	**Il contesto è il prodotto**	Più segnale nella finestra, migliori le risposte.
03	**Il terminale è l'interfaccia**	Nessuna UI nuova: i tuoi strumenti di sempre.

FONTI

https://opencode.ai/docs/

```
$ opencode
```

poi /init nella root del progetto